

ESP32 & MPU6050 Pedometer System

Embedded Firmware Architecture, Schematics & Dynamic Step Detection Algorithm

- PEDOMETER FIRMWARE
- ESP32 MICROCONTROLLER
- MPU6050 ACCELEROMETER
- SSD1306 OLED

1. Executive Overview

This technical document provides a comprehensive analysis of the Arduino C++ firmware designed for real-time step counting (pedometer) using an **ESP32 microcontroller**, an **MPU6050 6-axis IMU sensor**, and an **SSD1306 OLED display**.

During human locomotion (walking or running), the foot's impact with the ground produces periodic peaks in 3D acceleration. The system measures dynamic 3-axis acceleration vectors, derives the total magnitude, and applies a combined **Threshold Crossing** and **Rising Edge Detection Algorithm** with timed debouncing to count user steps accurately while filtering out ambient vibrations.

2. System Wiring & Schematic Diagram

The hardware uses dual hardware I2C channels on the ESP32 to communicate separately with the SSD1306 display and MPU6050 accelerometer sensor.

Circuit Schematic & Wiring Layout



COMPONENT	INTERFACE BUS	SIGNAL / PIN	ESP32 GPIO PIN	DESCRIPTION
MPU6050 Accelerometer	Hardware I2C-1 (MPUWire)	SDA	GPIO 21	Data line for sensor communications
		SCL	GPIO 20	Clock line for sensor communications
		VCC	3V3	3.3V Power supply input
		GND	GND	System ground reference
SSD1306 OLED (128x64)	Hardware I2C-0 (Wire)	SDA	GPIO 21	Data line for display buffer transfer
		SCL	GPIO 20	Clock line for display buffer transfer

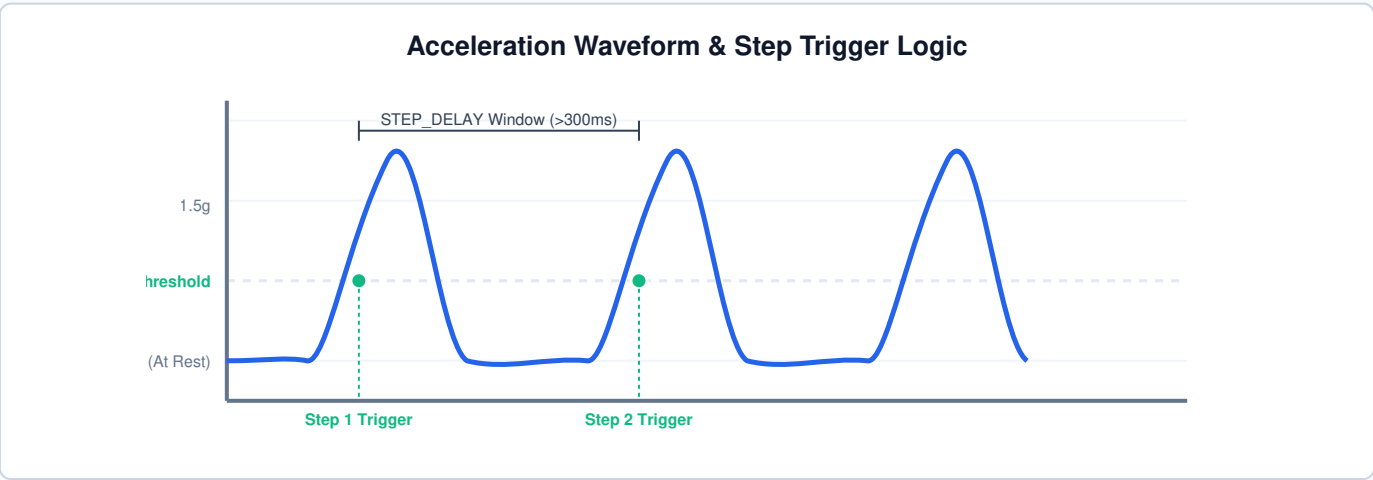
COMPONENT	INTERFACE BUS	SIGNAL / PIN	ESP32 GPIO PIN	DESCRIPTION
		VCC	3V3	3.3V Power supply input
		GND	GND	System ground reference

3. Mathematical Model of Step Detection

During human walking, the acceleration along the individual X, Y, and Z axes varies depending on the wearable device's physical orientation. To make step counting independent of sensor mounting angle, the algorithm computes the **Total Vector Magnitude** (A_{total}) measured in gravitational units (g):

$$A_{total} = \sqrt{(a_x^2 + a_y^2 + a_z^2)}$$

When standing still, Earth's gravity exerts a constant static load: $A_{total} \approx 1.0g$. During a heel strike while walking, acceleration rises sharply and forms a dynamic wave peak.



4. Step Detection Algorithm Rules

To register a single step, three distinct conditions must be satisfied simultaneously in the firmware code:

Triple-Condition Step Logic

- Threshold Exceeded:** `totalAcceleration > STEP_THRESHOLD` ($1.20g$). Ensures small movements do not register.
- Rising Edge Detection:** `previousAcceleration <= STEP_THRESHOLD`. Confirms acceleration is moving upward across the threshold, ensuring each peak is counted only once.
- Debounce Time Delay:** `currentTime - lastStepTime > STEP_DELAY` ($300ms$). Prevents double-counting from secondary bounce vibrations.

5. Firmware Parameter Configuration Matrix

CONSTANT / VARIABLE	CONFIGURED VALUE	UNIT	ALGORITHM RATIONALE
STEP_THRESHOLD	1.20	g	Minimum acceleration magnitude required to identify heel strike peak.

CONSTANT / VARIABLE	CONFIGURED VALUE	UNIT	ALGORITHM RATIONALE
STEP_DELAY	300	ms	Minimum duration between human steps (max allowable pace: ~200 steps/min).
previousAcceleration	1.0	g	Stores prior frame reading to evaluate rising edge ($A_{prev} \leq Threshold < A_{curr}$).
lastStepTime	0	ms	Timestamp tracking when the last step occurred for timing calculation.
stepCount	0	count	Integer variable storing total step tally shown on OLED.

6. Step-by-Step Code Execution Breakdown

A. Sensor Read & Scale Factor Conversion

The function `readAcceleration()` reads 6 raw bytes from register `ACCEL_XOUT_H` over I2C. The two 8-bit bytes for each axis are combined into a 16-bit signed integer and divided by the sensitivity scale factor (`16384.0 LSB/g` for MPU6050 default $\pm 2g$ range):

```
void readAcceleration(float &ax, float &ay, float &az) {
  MPUWire.beginTransaction(MPU6050_ADDR);
  MPUWire.write(ACCEL_XOUT_H);
  MPUWire.endTransmission(false);

  MPUWire.requestFrom(MPU6050_ADDR, 6, true);

  int16_t rawX = (MPUWire.read() << 8) | MPUWire.read();
  int16_t rawY = (MPUWire.read() << 8) | MPUWire.read();
  int16_t rawZ = (MPUWire.read() << 8) | MPUWire.read();

  // MPU6050 default +-2g range -> 16384 LSB/g
  ax = rawX / 16384.0;
  ay = rawY / 16384.0;
  az = rawZ / 16384.0;
}
```

B. Dynamic Peak Detection & Loop Implementation

In the main loop, magnitude is calculated using `sqrt()`. The conditional check verifies the threshold crossing and timing before incrementing `stepCount` and refreshing the OLED screen:

```

void loop() {
    float ax, ay, az;
    readAcceleration(ax, ay, az);

    // Calculate 3D magnitude
    float totalAcceleration = sqrt(ax * ax + ay * ay + az * az);
    unsigned long currentTime = millis();

    // Evaluate step conditions
    if (
        totalAcceleration > STEP_THRESHOLD &&
        previousAcceleration <= STEP_THRESHOLD &&
        currentTime - lastStepTime > STEP_DELAY
    ) {
        stepCount++;
        lastStepTime = currentTime;

        Serial.print("Step detected! Steps = ");
        Serial.println(stepCount);

        displaySteps(); // Update OLED GUI
    }

    previousAcceleration = totalAcceleration; // State update
    delay(50); // 20 Hz loop rate
}

```

7. Hardware Performance & Enhancements

- **Adaptive Peak Thresholding:** The fixed threshold (**1.20g**) works well for normal walking. For running vs. slow walking, implementing a dynamic exponential moving average (EMA) threshold improves accuracy across varying movement speeds.
- **Complementary/Kalman Filtering:** Adding low-pass filtering on A_{total} eliminates high-frequency sensor noise from mechanical impacts.
- **Power Efficiency:** Delaying screen redraws until steps occur saves CPU overhead and reduces power consumption.